

Operations on Data Structures

Q. Discuss the various fundamental operations that can be performed on data structures, explaining each with a suitable example.

> **Definition to Remember**

The Six Fundamental Operations

| Operation | Definition | Example |

Traversal Algorithm Example (Array)

```text

## Trade-offs Between Data Structures

| DS | Fast Operations | Slow Operations |

---

> **Must-Write Points (for 10 marks)**

---

> **Quick Recall**

> `Tra  
structu

---

## Algorithmic Implementations & Executable C Codes

### 1. Array Operations: Traversal, Insertion & Deletion (C Code)

```
```c
```

#includ

```
void traverse(int arr[], int n) {
```

printf("

```
int insertElement(int arr[], int n, int capacity, int element, int pos) {
```

if (n >=

```
int deleteElement(int arr[], int n, int pos) {
```

if (pos

```
int main() {
```

int arr[

```
printf("--- Initial Array ---\n");
```

travers

```
printf("\n--- Inserting 25 at Index 2 ---\n");
```

```
n = ins
```

```
printf("\n--- Deleting Element at Index 3 (30) ---\n");
```

```
n = de
```

```
return 0;
```

```
}
```

```
---
```

2. Searching Algorithms: Linear Search vs. Binary Search (C Code)

```
```c
```

```
#includ
```

```
// Linear Search - O(n)
```

```
int line
```

```
// Binary Search - O(log n) [Requires Sorted Array]
```

```
int bina
```

```
int main() {
```

```
int arr[
```

```
int linRes = linearSearch(arr, n, target);
```

```
printf("
```

```
int binRes = binarySearch(arr, n, target);
```

```
printf("
```

```
return 0;
```

```
}
```

```

```

### 3. Sorting & Merging Operations (C Code)

```
```\n#include <stdio.h>\n\n// Bubble Sort Operation - O(n^2)\nvoid bubbleSort(int arr[], int n)\n{\n    for (int i = 0; i < n - 1; i++)\n        for (int j = 0; j < n - i - 1; j++)\n            if (arr[j] > arr[j + 1])\n                swap(&arr[j], &arr[j + 1]);\n}\n\n// Merging Two Sorted Arrays - O(m + n)\nvoid mergeArrays(int A[], int m, int B[], int n, int C[])\n{\n    int i = 0, j = 0, k = 0;\n    while (i < m & j < n)\n    {\n        if (A[i] < B[j])\n            C[k++] = A[i++];\n        else\n            C[k++] = B[j++];\n    }\n    while (i < m)\n        C[k++] = A[i++];\n    while (j < n)\n        C[k++] = B[j++];\n}\n\nint main() {\n    int A[] = {5, 12, 22, 25, 34, 44};\n    int B[] = {1, 3, 8, 12, 15, 20, 28, 30, 35, 40};\n    int C[16];\n\n    mergeArrays(A, m, B, n, C);\n\n    printf("Merged Array: ");\n    for (int i = 0; i < 16; i++)\n        printf("%d ", C[i]);\n    printf("\\n");\n\n    return 0;\n}
```